

Лабораторна робота № 6

з курсу “Інтелектуальний аналіз даних”

ТЕМА: Застосування дерев класифікації. Класифікація за допомогою дерев рішень

META: вивчення методики проведення класифікації даних за допомогою використання дерев рішень на прикладі розв'язання конкретних завдань ІАД

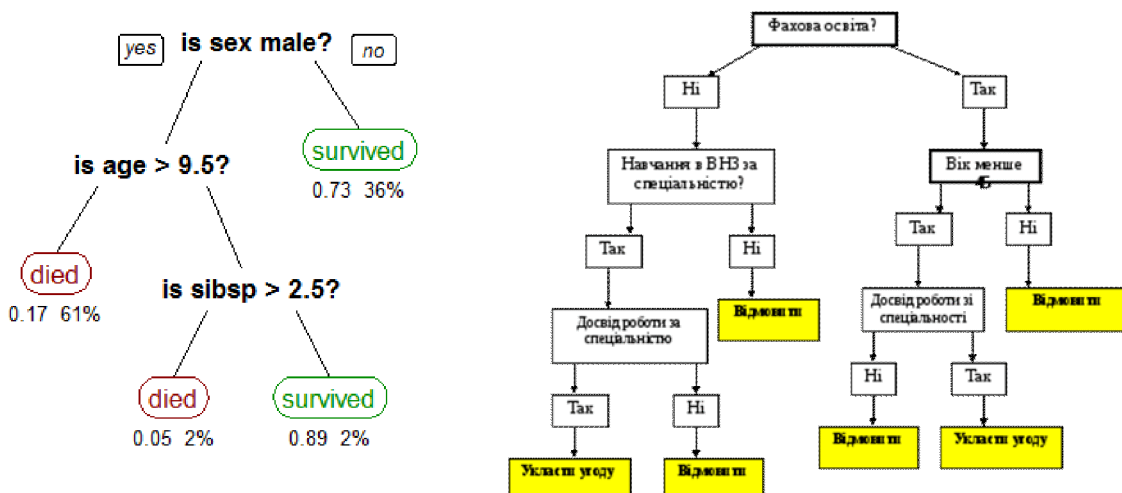
Теоретичні відомості

Дерева рішень використовуються в задач класифікації, які характеризуються високою відповідальністю: діагностиці, медицині, фінансах. Два найпопулярніших алгоритми побудови дерев - CART и C4.5.

Дерева рішень, використовувані в Data Mining, бувають двох основних типів:

- Аналіз дерева класифікації, коли прогнозований результат є класом, до якого належать дані;
- Регресійний аналіз дерева, коли прогнозований результат можна розглядати як дійсне число (наприклад, ціна на будинок, або тривалість перебування пацієнта в лікарні).

Зупинимось на першому типі. Вони являють собою ієрархічний набір правил типу “Якщо – тоді –інакше”, що має вигляд дерева. Корінь дерева є фіктивним; він позначає початок процесу класифікації. Листки дерева відповідають класам рішень. Проміжні вершини відповідають логічним умовам, за якими аналізуються ознаки об'єкта класифікації. Щоб визначити клас належності деякого об'єкту, потрібно відповісти для нетермінальних вершин дерева, “значення параметра x більше A ?”. Якщо відповідь «так», здійснюється перехід до правої вершини наступного рівня дерева, якщо «ні» – до лівої вершини. Перехід по дереву продовжується до тих пір, поки не потрапимо на один із його листків.



На сьогоднішній день існує велика кількість алгоритмів, що реалізують дерева рішень: CART, C4.5, CHAID, CN2, NewId, ITrule і інші. Розглянемо алгоритм CART (Classification and Regression Trees), що призначений для побудови дерева рішень. Дерева рішень, побудовані за допомогою CART, є **бінарними**, тобто містять тільки два нащадка в кожному вузлі.

Він рекурсивно ділить початковий набір даних на підмножини, які стають все більш і більш гомогенними щодо певних ознак, в результаті чого формується деревоподібна ієрархічна структура. Розподіл здійснюється на основі традиційних логічних правил виду "ЯКЩО (A) ТО (B)", де A - деяка логічна умова, а B - процедура розбиття підмножини на дві частини, для однієї з яких умова A істинна, а для іншої - хибна.

Загальне правило вибору опорного значення для кожного вузла побудованого дерева можна сформулювати наступним чином: **"обрана ознака повинна розбити множину X^* так, щоб одержувані в результаті підмножини X^*k , $k = 1, 2, \dots, p$, склалися з спостережень, що належать до одного класу, або були максимально наближені до цього"**.

Неможливо підібрати об'єктивний внутрішній критерій, що приводить до хорошого компромісу між безпомилковістю і компактністю моделі, тому стандартний механізм оптимізації дерев заснований на перехресній перевірці. Для цього навчальна вибірка поділяється, наприклад, на 10 рівних частин: 9 частин використовуються для побудови дерева, а решта грає роль тестувальної вибірки. Після багаторазового повторення цієї процедури з деякого набору дерев-претендентів з прийнятим (для розв'язуваної задачі) розкидом значень того чи іншого критерію якості моделі, вибирається дерево, яке показало найкращий результат в ході перехресної перевірки.

Функція `gpart()` з однойменного пакету виконує рекурсивний вибір для кожного наступного вузла таких опорних значень, які призводять до мінімальної суми квадратів внутрішньогрупових відхилень D_t для всіх t вузлів дерева. Для оцінки якості побудованого дерева T в ході його оптимізації використовується наступна сукупність критеріїв:

- $CC(T) = \sum(D_t + \lambda t)$ - штраф на складність моделі (cost complexity), що включає штрафний множник λ за кожну невідсічену гілку; $\square D_0$ - девіанс для нульового дерева (тобто оцінка розкиду у вихідних даних);
- $C_p = \lambda / D_0$ - відносний штраф на складність моделі (також відомий як "статистика Меллоуза", Mallows's C_p);
- $RE_{Ler} = \sum D_t / D_0$ - відносна помилка навчання для дерева з t вузлів;
- SE - стандартне відхилення для помилки перехресної перевірки.

Оптимальним зазвичай вважається дерево, що складається з такої кількості гілок t , для якого сума $(CVer + SE)$ є мінімальною.

Як приклад розглянемо побудову в R-studio дерева CART, що прогнозує велику кількість водоростей групи A1 в залежності від гідрохімічних показників води і умов

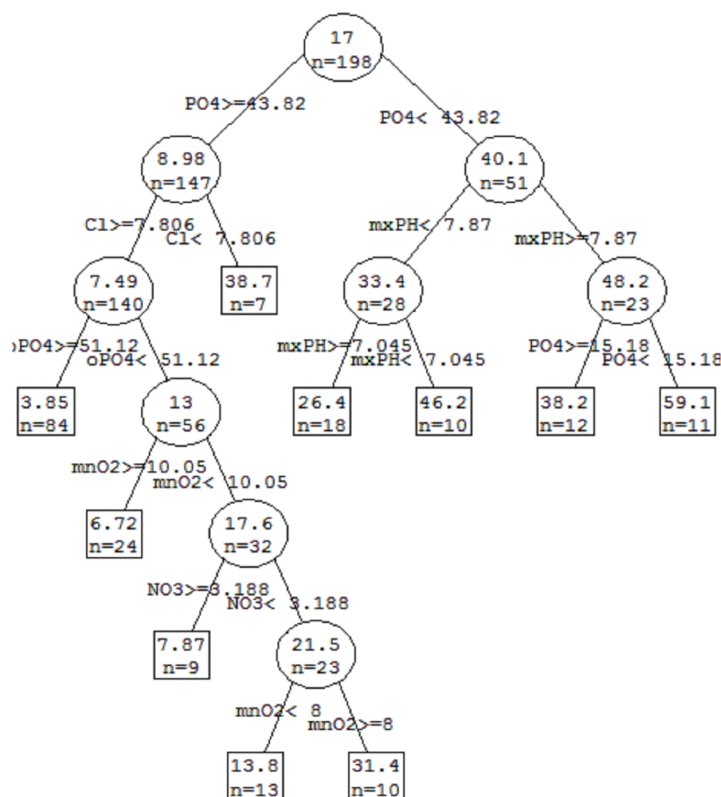
відбору проб в різних водотоках (дані містяться в файлі `algae.csv`). Використаємо бібліотеки `library(DMwR)` та `library(rpart)`.

Спочатку скористаємося пакетом `rpart`. Зазвичай при побудові дерев регресії застосовується двокрокова процедура: функція `rpart()` встановлює зв'язку між залежною і незалежними змінними і формує бінарне дерево, а функція `prun()` виконує обрізання зайвих гілок:

```
algae<-read.csv("F:/lab1/algae.csv")
str(algae) # перевіряємо структуру файлу
algae<- algae[-manyNAs(algae), ] #видаляємо два записи з NA
(rt.al <- rpart(al ~ ., data = algae[, 1:12]))
```

Приведеною вище командою ми побудували повне дерево з 9 вузлів і 10 листів, позначених символом *. У кожному рядку представлені по порядку: умова розбиття, число спостережень, відповідних цій умові, девіанс (в даному випадку - це еквівалент суми квадратів відхилень від групового середнього) і середнє значення відгуку для виділеної гілки. Наприклад, перед першою розбивкою загальна множина з 198 спостережень має середнє значення $m = 16.99$ при девіансе $D = 90401$. При $PO4 \geq 43.82$ ця множина ділиться на дві частини: 2) 147 спостережень ($m = 8.98$, $D = 31279$) і 3) 51 спостереження з високим рівнем достатку водоростей ($m = 40.1$, $D = 22442$). Подальші розбиття кожної з цих двох частин аналогічні.

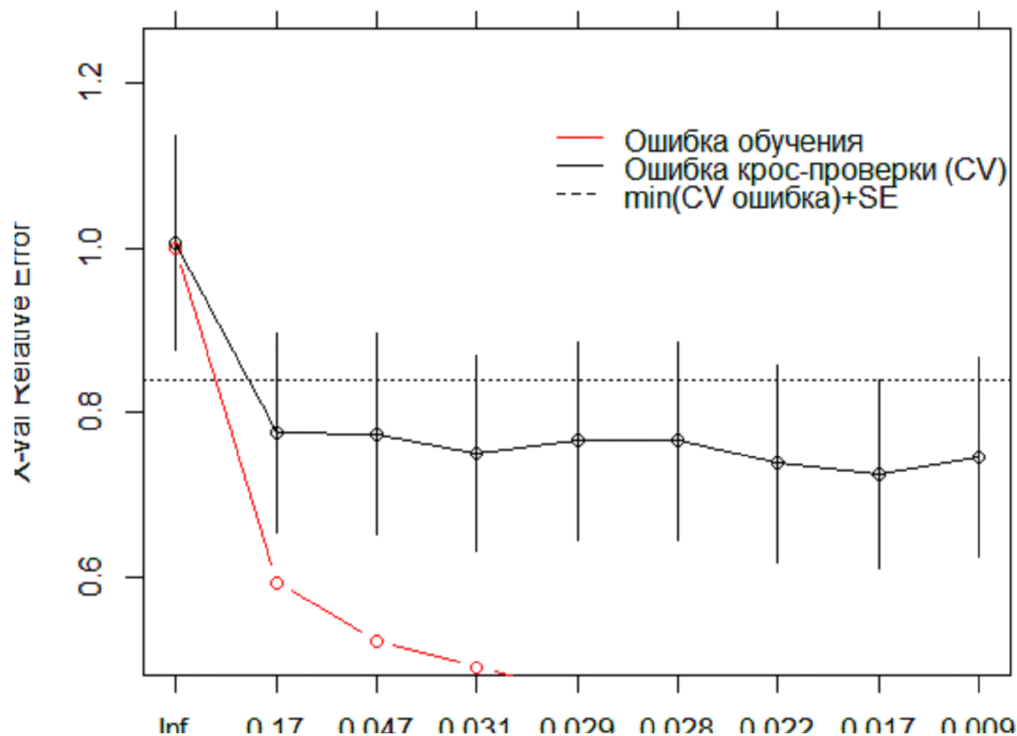
Для графічного представлення отриманого дерева можна використати функції: `plot()`, `prettyTree()` з пакету `DMwR` і `prp()` з пакета `rpart.plot` (рис. 2):



```
prettyTree(rt.a1, compress = TRUE)
prp(rt.a1, compress = TRUE)
```

Функція `rpart()` і інші функції з цього пакету мають власні можливості виконати перехресну перевірку і оцінити її помилку при різних значеннях штрафу на складність моделі (рис. 2):

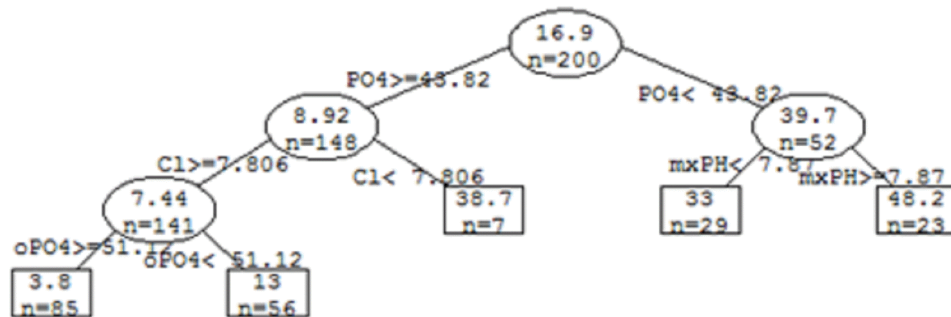
```
rtp.a1 <- rpart(a1 ~ ., data = algae[, 1:12], control =
rpart.control(cp = .005))
plotcp(rtp.a1)
with(rtp.a1, {lines(cptable[, 2] + 1, cptable[, 3],
type = "b", col = "red")
legend(locator(1), c("помилка навчання", "помилка крос-
перевірки (CV)", "min(CV помилка)+SE"),
lty = c(1, 1, 2), col = c("red", "black", "black"), bty =
"n")
```



На графіку видно, що мінімум відносної помилки при перехресній перевірці приблизно припадає на значення $cp = 0.029$. Виконаємо обрізку дерева при цьому значенні (рис. 3):

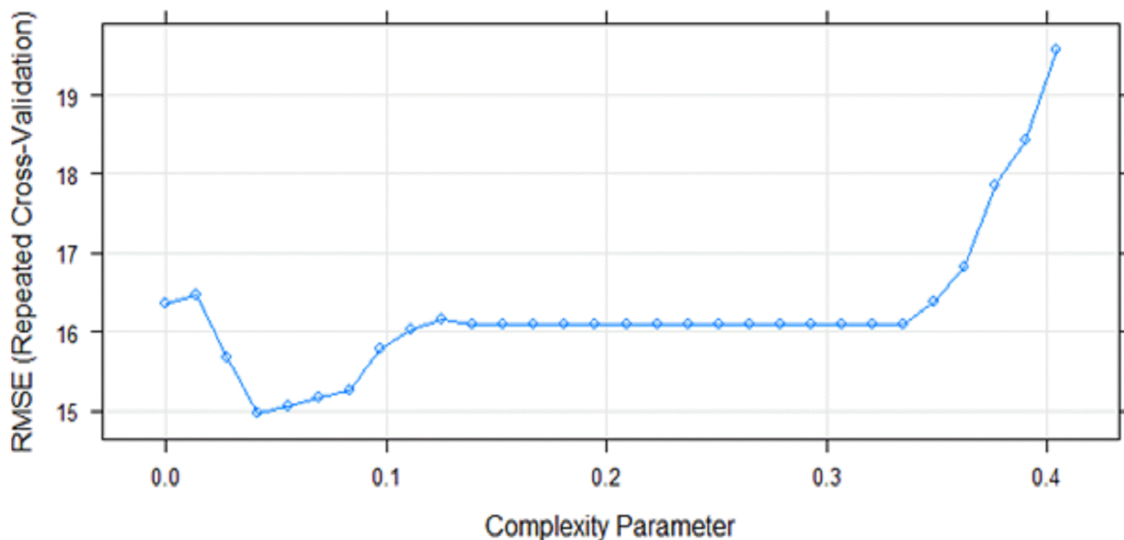
```
rtp.a1 <- prune(rtp.a1, cp = 0.029)
prettyTree(rtp.a1)
```

```
prettyTree(rtp.a1)
```



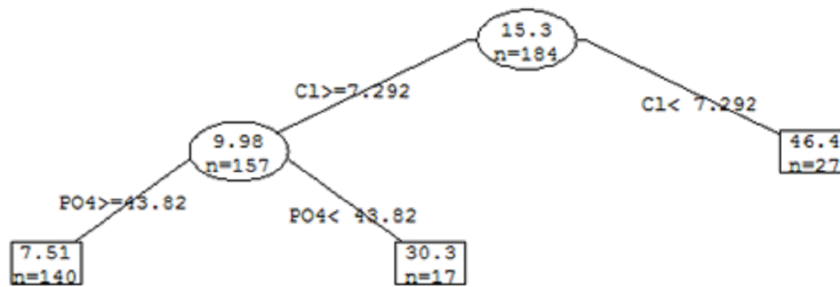
Виконаємо додаткову оптимізацію параметра cp з використанням функції `train()` з пакету `caret`. Будемо тестувати дерева регресії при 30 значеннях критерію cp , застосовуючи до кожного з них 10-кратну перехресну перевірку з 3 повторних (рис. 4)

```
install.packages("caret")
library(caret)
cvCtrl<- trainControl(method = "repeatedcv", repeats = 3)
rt.a1.train <- train(a1 ~ ., data = na.omit(algae[, 1:12]),
method = "rpart", tuneLength = 30, trControl = cvCtrl)
plot(rt.a1.train)
```



Побудуємо результуюче дерево рішень (після відсікання) (рис.5)

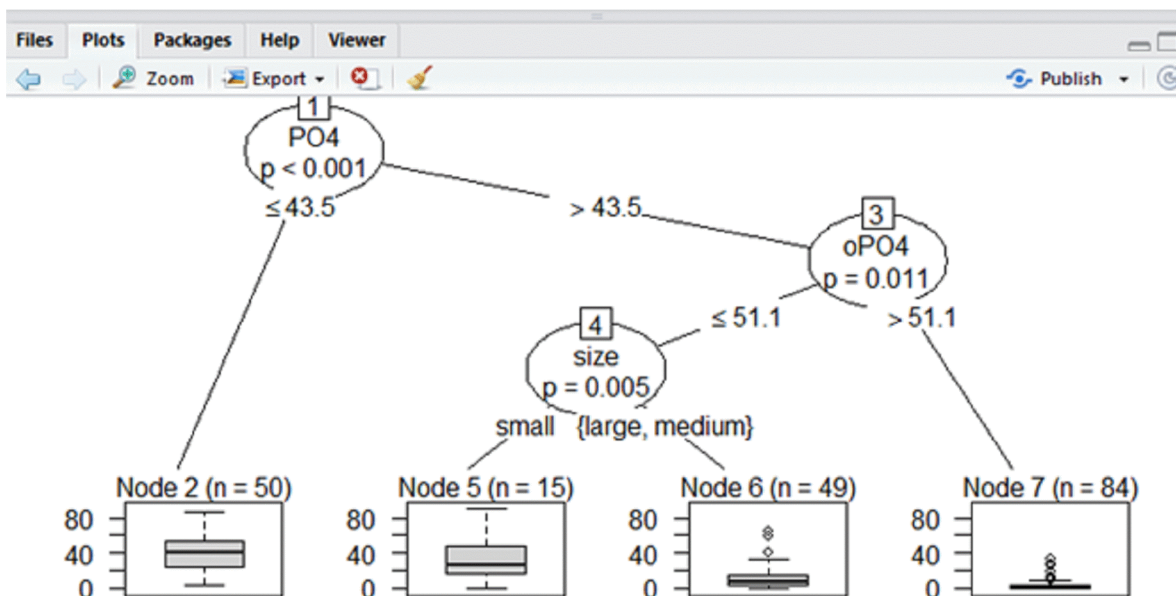
```
rtt.a1 <- rt.a1.train$finalModel
prettyTree(rtt.a1)
```



Таким чином, при $\alpha = 0.0418$ було отримано суттєво урізане дерево, яке, правда, значно втратило у своїй цінності з точки зору біологічних механізмів, що визначають різноманіття водоростей.

Також, для побудови дерева класифікації можна скористатись функцією `ctree()` бібліотеки `party`, що використовує метод побудови дерев на основі "умовного виведення" (рис. 6):

```
library(party) # використання методу "умовного виведення"
(ctree.a1 <- ctree(a1 ~ ., data = algae[, 1:12]))
plot(ctree.a1)
```



Можна з розумною обережністю зробити висновок про те, що прогнозуюча цінність дерев після додаткової оптимізації функцією `train()` дещо зростає, а дерева, побудовані за допомогою `rpart()`, точніше, ніж дерева "умовного виведення" `ctree()`.

Завдання на лабораторну роботу

1. Розглянути наведені в теоретичних відомостях методи і функції побудови та дослідження дерев рішень. За допомогою вбудованої довідки вивчити параметри функцій `rpart()`, `prun()`, `ctree()`, `prettyTree()` та `prp()`.

2. Для масиву даних, що містяться в файлі `algae.csv` провести побудову дерева рішень, використовуючи відповідні стандартні функції мови R .

3. Провести дослідження побудованого дерева рішень. Дослідити залежність помилок класифікації від кількості рівнів дерева рішень. Виконати відсікання дерева за вузлами та за рівнями і провести порівняння отриманих результатів.

4. В файлі “iris.dat” у вигляді 4-вимірної масиви задано опис 3-х класів ірисів. провести класифікацію даних та побудувати дерево рішень. Оцінити точність класифікації даним класифікатором

5. Провести класифікацію деякого набору даних з сайту <http://data.worldbank.org>.
//додатково

Вимоги до оформлення звіту

Захист лабораторної роботи проводиться при наявності звіту. Захист роботи виконується у встановлені терміни. Звіт повинен бути оформлений на папері стандартного формату А4

Звіт повинен містити такі розділи:

1. Постановку задачі, розв’язування якої виконується на лабораторній роботі.
2. Короткий опис основних теоретичних положень та алгоритмів, що застосовуються для розв’язування поставленої задачі та будуть використані при реалізації завдань.
3. Контрольні приклади та результати їх виконання. Викладач може запропонувати свої приклади для перевірки працездатності програми.